In this chapter we'll be focusing on these issues, from the high-level (but fundamental) considerations of how many threads to use, which code to execute on which thread, and how this can affect the clarity of the code, to the low-level details of how to structure the shared data for optimal performance.

Let's start by looking at techniques for dividing work between threads.

8.1. Techniques for dividing work between threads

Imagine for a moment that you've been tasked with building a house. In order to complete the job, you'll need to dig the foundation, build walls, put in plumbing, add the wiring, and so forth. Theoretically, you could do it all yourself with sufficient training, but it would probably take a long time, and you'd be continually switching tasks as necessary. Alternatively, you could hire a few other people to help out. You now have to choose how many people to hire and decide what skills they need. You could, for example, hire a couple of people with general skills and have everybody chip in with everything. You'd still all switch tasks as necessary, but now things can be done more quickly because there are more of you.

Alternatively, you could hire a team of specialists: a bricklayer, a carpenter, an electrician, and a plumber, for example. Your specialists just do whatever their specialty is, so if there's no plumbing needed, your plumber sits around drinking tea or coffee. Things still get done quicker than before, because there are more of you, and the plumber can put the toilet in while the electrician wires up the kitchen, but there's more waiting around when there's no work for a particular specialist. Even with the idle time, you might find that the work is done faster with specialists than with a team of general handymen. Your specialists don't need to keep changing tools, and they can probably each do their tasks quicker than the generalists can. Whether or not this is the case depends on the particular circumstances—you'd have to try it and see.

Even if you hire specialists, you can still choose to hire different numbers of each. It might make sense to have more bricklayers than electricians, for